

Lab11

Bhanu Prasad cs24b059

April 2026

1 Introduction

The aim of this lab is to address control hazards in a two-stage pipelined processor consisting of Stage 1 (Fetch/Decode/Register Read) and Stage 2 (Execute/Writeback). Branch instructions can interrupt the smooth flow of the pipeline, causing stalls.

To minimize this issue, a dynamic branch prediction mechanism was introduced. The design uses a 64-entry Branch History Table (BHT), where each entry holds a 2-bit saturating counter that predicts whether a branch will be taken or not taken based on past executions.

Since decoding completes early in Stage 1, the branch target address can be directly calculated using the PC and immediate value. This removes the necessity of using a Branch Target Buffer (BTB).

2 Modifications

The following changes were implemented to include branch prediction in the pipeline:

- **Stage 1 (Fetch + Decode) - Prediction Logic:**

- A 64-entry BHT was implemented using memory, indexed with $PC[7 : 2]$.
- Each entry provides a 2-bit value, and the prediction is determined using:

$$prediction = MSB(BHT[PC[7 : 2]])$$

- If the prediction indicates "Taken", the next PC is set to $PC + Imm_B$. Otherwise, execution proceeds with $PC + 4$.

- **Stage 2 (Execute + Resolve) - Outcome Evaluation:**

- The actual branch condition is checked in this stage.
- Misprediction is detected using:

$$mispredict = prediction \oplus actualoutcome$$

- A value of 1 indicates an incorrect prediction, while 0 indicates correctness.
- **Handling Mispredictions:**
 - If a misprediction occurs, the incorrect instruction in Stage 2 is flushed.
 - The PC is updated with the correct target address to resume proper execution.
- **Updating the BHT:**
 - The BHT entry is updated based on the actual outcome.
 - The counter is incremented for taken branches and decremented for not taken branches, while ensuring it remains within valid limits.

3 Testing and Analysis

The design was tested using the provided benchmark on both the original pipeline and the modified version with branch prediction.

- **Baseline Pipeline Cycles:** 1183714
- **Modified Pipeline Cycles:** 779056

Performance Evaluation:

- **Cycles Reduced:** 404658
 - **Improvement:** 34.19%
- (Computed using $\frac{Baseline - Modified}{Baseline} \times 100$)

Discussion:

The enhanced pipeline demonstrates noticeable performance gains. In the original design, branch instructions frequently caused stalls due to control hazards.

With the addition of branch prediction, the processor adapts to recurring branch patterns. This is particularly beneficial in loop-heavy code, where predictions become more accurate over time. As a result, fewer mispredictions occur, reducing pipeline flushes and improving overall efficiency.

4 Conclusion

The dynamic branch predictor significantly improves pipeline performance by reducing control hazards. Using a 64-entry BHT provides a good balance between hardware complexity and performance improvement.

The experiment confirms that even a simple prediction strategy can lead to substantial gains by minimizing unnecessary stalls.

5 Effect of Increasing BHT Size

To study the impact of predictor capacity, the BHT size was increased from 64 entries to 128 and 256 entries. The indexing scheme was adjusted as follows:

- 64 entries: $PC[7 : 2]$
- 128 entries: $PC[8 : 2]$
- 256 entries: $PC[9 : 2]$

5.1 Observed Results

- **64-entry BHT:** 779056 cycles
- **128-entry BHT:** 778878 cycles
- **256-entry BHT:** 778880 cycles

5.2 Analysis

Increasing the BHT size helps reduce aliasing, where multiple branches share the same entry. Moving from 64 to 128 entries slightly improves prediction accuracy, leading to a small performance gain.

However, increasing the size further to 256 entries does not provide any meaningful improvement. This suggests that most active branches are already effectively captured with 128 entries.

5.3 Conclusion of Experiment

The results indicate that while increasing BHT size can improve performance, the benefit plateaus beyond a certain point. This demonstrates diminishing returns and highlights the importance of selecting an optimal predictor size that balances cost and performance.